

Chapter 01

Python Decorators

Understanding closures, function wrapping, class-based decorators, and the practical patterns behind logging, timing, and retry logic.

WHAT THIS CHAPTER COVERS

- ▶ First-class functions & closures recap
- ▶ Anatomy of a decorator
- ▶ Handling *args and **kwargs
- ▶ Class-based decorators
- ▶ Practical: logging & timing decorators
- ▶ Chaining decorators correctly
- ▶ functools.wraps and why it matters
- ▶ Parametrized decorators & @property

Python Decorators

1. Prerequisite Recap: First-Class Functions & Closures

Definition

First-class functions means functions in Python can be treated like any other object — passed as arguments, returned from other functions, and assigned to variables. A **closure** is an inner function that remembers and has access to variables from its enclosing (outer) function's scope, even after the outer function has finished executing.

Why It Exists

Decorators are built entirely on top of these two ideas. A decorator is essentially a specialized closure: an outer function receives another function as input, defines an inner function that "remembers" the input function, and returns that inner function instead of executing it immediately.

Example

```
def outer_function(msg):
    # msg is a "free variable" for inner_function below
    def inner_function():
        print(msg)
    return inner_function    # returned WITHOUT executing it

hi_func = outer_function("Hi")
bye_func = outer_function("Bye")

hi_func()    # prints "Hi"
bye_func()   # prints "Bye"
```

Each call to `outer_function` creates a fresh scope. `inner_function` is returned unexecuted, and it keeps a private reference to whatever `msg` was at the time it was created — that persistent reference is the closure.

IMPORTANT

A decorator does not modify the source code of the original function. It wraps new behavior around it and returns a new callable — the original function's code is never edited.

🔗 Quick Revision — First-Class Functions & Closures

- First-class functions can be passed, returned, and assigned like any other value.
- A closure is an inner function that remembers variables from its enclosing scope.
- Returning a function without parentheses returns it *unexecuted*, ready to be called later.
- Every decorator is built on the closure pattern: outer function in, inner function out.

2. What Is a Decorator?

Definition

A *decorator* is a function that takes another function as an argument, adds some kind of functionality to it, and returns another function — all without altering the source code of the original function.

How It Works

1. The decorator function accepts the target function as a parameter (conventionally called `original_function`).
2. Inside the decorator, an inner `wrapper` function is defined. This wrapper calls `original_function` and can run additional code before and/or after that call.
3. The decorator returns `wrapper` (unexecuted) instead of the original function.
4. The caller now holds a reference to `wrapper`, so every "call" to the original function actually goes through the wrapper first.

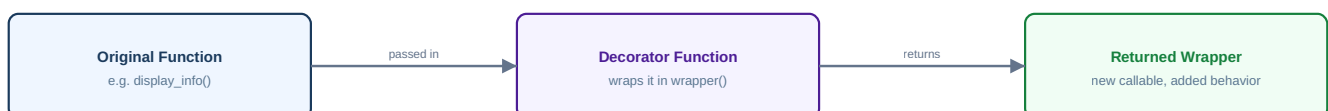


Figure 1.1 — A decorator receives a function, wraps it, and returns a new callable.

Example

```
def decorator_function(original_function):
    def wrapper():
        return original_function()
    return wrapper

def display():
    print("display function ran")

decorated_display = decorator_function(display)
decorated_display()  # -> "display function ran"
```

THE @ SYNTAX

Placing `@decorator_function` directly above a function definition is exactly equivalent to reassigning the function to the decorator's return value:

```
@decorator_function
def display():
    print("display function ran")

# is identical to:
def display():
    print("display function ran")
display = decorator_function(display)
```

INTERVIEW TIP

Be ready to state the decorator definition precisely: "a function that takes a function, adds functionality, and returns a function, without modifying the original function's source code." Interviewers often check whether you understand that the `@` syntax is just sugar for reassignment.

Quick Revision — What Is a Decorator?

- A decorator = function in, function out, extra behavior added in between.
- The `@decorator` syntax above a function is equivalent to `func = decorator(func)`.
- The wrapper function is what actually gets called after decoration.
- Decorators never edit the original function's source code.

3. Handling Arguments with `*args` and `**kwargs`

Problem It Solves

The simple decorator in Section 2 breaks if the decorated function accepts parameters, because `wrapper()` was defined with no parameters. Calling a decorated function that takes arguments raises a `TypeError`.

COMMON MISTAKE

Writing `def wrapper():` with no parameters, then decorating a function like `display_info(name, age)`. This throws a `TypeError: wrapper() takes 0 positional arguments but 2 were given`.

How It Works

Give the wrapper function `*args` and `**kwargs`, and forward them into the call to `original_function`. This lets a single decorator work with *any* function signature.

```
def decorator_function(original_function):
    def wrapper(*args, **kwargs):
        return original_function(*args, **kwargs)
    return wrapper

@decorator_function
def display_info(name, age):
    print("display_info ran with arguments ({}, {})".format(name, age))

display_info("John", 25)
```

`*args` collects any number of positional arguments into a tuple; `**kwargs` collects any number of keyword arguments into a dictionary. Forwarding both means the decorator is reusable across functions with completely different signatures.

Quick Revision — `*args` and `**kwargs`

- A wrapper with no parameters only works on zero-argument functions.
- `*args` captures positional arguments as a tuple; `**kwargs` captures keyword arguments as a dict.
- Forward both into `original_function(*args, **kwargs)` so any signature works.
- This is what makes a decorator generically reusable across many functions.

4. Class-Based Decorators

Definition

Instead of writing a decorator as a function, it can be written as a class. The class uses `__init__` to capture the original function and `__call__` to define what happens when the "decorated" object is invoked — mirroring the outer-function/wrapper-function pattern.

How It Works

```
class DecoratorClass:
    def __init__(self, original_function):
        self.original_function = original_function # ties function to instance

    def __call__(self, *args, **kwargs):
        print("call method executed this before", self.original_function.__name__)
        return self.original_function(*args, **kwargs)

@DecoratorClass
def display_info(name, age):
    print("display_info ran with arguments ({}, {})".format(name, age))

display_info("John", 25)
```

MAPPING BETWEEN THE TWO STYLES

Function-based decorator	Class-based decorator
Outer function parameter	<code>__init__(self, original_function)</code>
Inner <code>wrapper()</code> function	<code>__call__(self, *args, **kwargs)</code>
<code>return wrapper</code>	The class instance itself becomes callable

REAL WORLD INSIGHT

Function-based decorators are far more common in everyday Python code and in most third-party libraries, but class-based decorators are useful when the decorator needs to maintain more complex internal state across calls.

Quick Revision — Class-Based Decorators

- `__init__` captures and stores the original function on the instance.
- `__call__` replaces the role of the `wrapper()` function.
- Both styles produce functionally identical results.
- Function-based decorators remain the more common convention in practice.

5. Practical Decorators: Logging and Timing

Why They Exist

Two of the most common real-world uses of decorators are logging function calls and measuring execution time — both are cross-cutting concerns that would otherwise need to be copy-pasted into every function.

1. LOGGING DECORATOR

```
def my_logger(original_function):
    import logging
    logging.basicConfig(filename='{}.log'.format(original_function.__name__),
                        level=logging.INFO)

    def wrapper(*args, **kwargs):
        logging.info(
            "Ran with args: {} and kwargs: {}".format(args, kwargs))
        return original_function(*args, **kwargs)
    return wrapper
```

Every call to the decorated function writes a log entry (filename matching the function name) recording exactly what arguments it was called with — without touching the function's own code.

2. TIMING DECORATOR

```
def my_timer(original_function):
    import time

    def wrapper(*args, **kwargs):
        t1 = time.time()
        result = original_function(*args, **kwargs)
        t2 = time.time() - t1
        print("{} ran for {} secs".format(original_function.__name__, t2))
        return result
    return wrapper
```

The wrapper records a start timestamp, runs the original function, computes the elapsed time, prints it, and only then returns the original result — the timing logic never touches the wrapped function's internals.

BEST PRACTICE

Keep cross-cutting concerns like logging and timing inside a decorator rather than duplicating that code inside every function. One decorator can be reused anywhere it's needed, and the logic stays in exactly one place.

Quick Revision — Logging & Timing Decorators

- Logging decorators record calls and arguments to a log file without modifying business logic.
- Timing decorators wrap the original call with a start/end timestamp diff.
- Both follow the same shape: pre-call logic -> call original -> post-call logic -> return.
- Decorators avoid duplicating the same boilerplate across many functions.

6. Chaining Multiple Decorators & functools.wraps

How It Works

Multiple decorators can be stacked on a single function by listing them one above another. Decorators apply **bottom-up** (the one closest to the function runs first, wrapping the function first), but at call time, execution happens **outer-to-inner** (the topmost decorator's wrapper runs first).

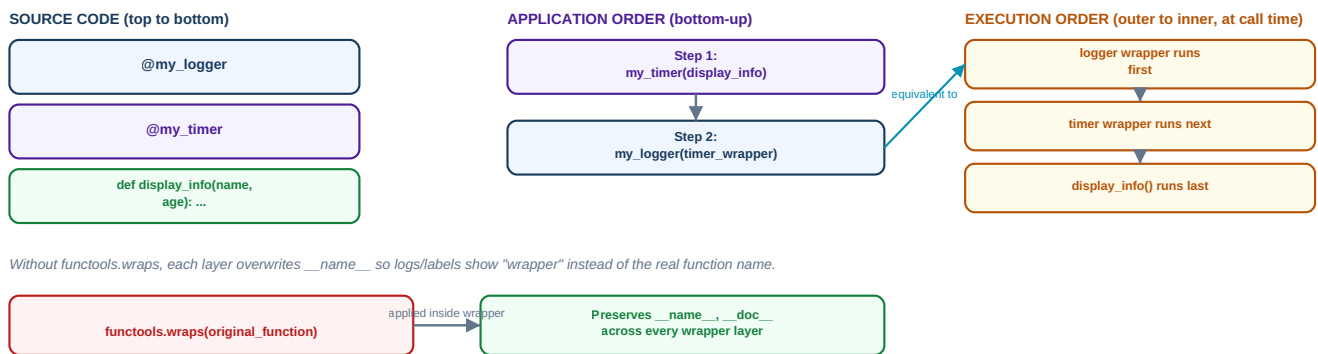


Figure 1.2 — Stacking order vs. execution order, and why `functools.wraps` is required.

Problem It Solves

When decorators are chained, the output of the first decorator (a **wrapper** function) becomes the input to the second decorator. Since `wrapper.__name__` is literally "wrapper", the second decorator loses all knowledge of the true original function's name — for example, a logging decorator ends up writing to `wrapper.log` instead of `display_info.log`.

COMMON MISTAKE

Stacking decorators without `functools.wraps` silently corrupts function metadata (`__name__`, `__doc__`) as it passes through each layer, breaking anything downstream that relies on the function's real identity, such as log filenames or introspection tools.

Fix: functools.wraps

```
from functools import wraps

def my_logger(original_function):
    import logging
    logging.basicConfig(filename='{}.log'.format(original_function.__name__),
                        level=logging.INFO)

    @wraps(original_function)
    def wrapper(*args, **kwargs):
        logging.info("Ran with args: {} and kwargs: {}".format(args, kwargs))
        return original_function(*args, **kwargs)
    return wrapper
```

Decorating the inner wrapper with `@wraps(original_function)` copies over the original function's `__name__`, `__doc__`, and other metadata onto the wrapper, so every layer of a decorator chain still reports the true, original function identity.

BEST PRACTICE

Always decorate the inner wrapper function with `@functools.wraps(original_function)` in every custom decorator you write — especially if it may ever be chained with other decorators.

Quick Revision — Chaining & functools.wraps

- Stacked decorators are applied bottom-up but execute outer-to-inner at call time.
- Each decorator layer wraps the previous layer's wrapper, not the true original function.
- Without `functools.wraps`, `__name__` becomes "wrapper" at every layer, breaking logs and introspection.
- `@wraps(original_function)` on the inner wrapper preserves the real function's identity through the whole chain.

7. Parametrized Decorators (Decorators That Accept Arguments)

Definition

A parametrized decorator is a decorator factory: an outer function that accepts configuration arguments (not the target function itself), and returns the actual decorator function, which in turn returns the wrapper. This adds one extra layer of nesting compared to a normal decorator.

Example: A Retry Decorator

```
from functools import wraps

def retry(n):
    # Layer 1: accepts the configuration argument (max attempts)
    def decorator_function(original_function):
        # Layer 2: the real decorator, receives the target function
        @wraps(original_function)
        def wrapper(*args, **kwargs):
            # Layer 3: the wrapper that runs on every call
            attempt = 1
            while attempt <= n:
                try:
                    result = original_function(*args, **kwargs)
                    return result          # success, exit immediately
                except Exception as e:
                    print("Retrying: Attempt ", attempt, "/", n)
                    attempt += 1
                    if attempt > n:
                        raise e          # out of attempts, propagate the error
            return wrapper
        return decorator_function

    @retry(3)
    def test():
        raise ValueError("Something went wrong")

    test()
```

Because `test()` always raises, `retry(3)` makes three attempts, prints a retry message after each failure, and re-raises the exception once the attempt count exceeds `n`. If `original_function` succeeds on any attempt, its result is returned immediately and the loop stops.

IMPORTANT

`@retry(3)` is not the decorator itself — calling `retry(3)` first *returns* the actual decorator (decorator_function), which is then applied to `test`. This is why a parametrized decorator needs three nested function levels instead of two.

Quick Revision — Parametrized Decorators

- A parametrized decorator needs three nested levels: config args -> decorator -> wrapper.
- `@retry(3)` first calls `retry(3)`, which returns the real decorator function.
- The retry pattern uses a loop with try/except to re-attempt on failure.
- Once max attempts are exhausted, the original exception is re-raised, not swallowed.

8. Built-In Decorators: @property, @x.setter, @x.deleter

Definition

@property is a built-in decorator that lets a method be accessed like a plain attribute (no parentheses needed). Paired with @<name>.setter and @<name>.deleter, it gives full control over how an attribute is read, assigned, and deleted, while callers still use ordinary attribute syntax.

Why It Exists

Without @property, computed values (like a full name built from first and last name) would have to be called as regular methods (emp.fullname()), and there would be no clean way to keep a derived attribute like email automatically in sync when the underlying first / last values change.

Example

```
class Employee:
    def __init__(self, first, last):
        self.first = first
        self.last = last

    @property
    def email(self):
        return f'{self.first}.{self.last}@email.com'

    @property
    def fullname(self):
        return '{} {}'.format(self.first, self.last)

    @fullname.setter
    def fullname(self, name):
        first, last = name.split(' ')
        self.first = first
        self.last = last

    @fullname.deleter
    def fullname(self):
        print("Deleted Name!")
        self.first = None
        self.last = None

emp1 = Employee('Tom', 'Hardy')
print(emp1.email)           # tom.hardy@email.com (no parentheses!)
emp1.fullname = 'Tom Holland' # invokes the setter
print(emp1.email)           # tom.holland@email.com – auto-updated
del emp1.fullname           # invokes the deleter
```

HOW THE THREE DECORATORS CONNECT

Decorator	Triggered By	Purpose
@property	emp1.fullname (read)	Defines the getter; makes the method act like an attribute.
@fullname.setter	emp1.fullname = "..."	Defines what happens on assignment.
@fullname.deleter	del emp1.fullname	Defines what happens on deletion.

INTERVIEW TIP

Be ready to explain that `@fullname.setter` only works because `fullname` was already turned into a `property` object by `@property` above it — the setter decorator is a method (`.setter`) that lives on that property object.

Quick Revision — @property Decorators

- `@property` turns a method into a read-only computed attribute (called without parentheses).
- `@.setter` lets you intercept and validate assignment to that attribute.
- `@.deleter` lets you intercept and customize behavior on `'del obj.attr'`.
- `email` in the example stays automatically in sync because it recomputes from `first/last` on every access.

K. Key Takeaways

- **Closures** are the foundation of decorators — an inner function retaining access to its enclosing scope's variables.
- **Decorators** take a function, wrap it with extra behavior, and return a new callable, without editing the original source code.
- ***args / **kwargs** in the wrapper make a decorator generically reusable across functions with any signature.
- **Class-based decorators** mirror the function-based pattern using `__init__` and `__call__`.
- **Logging and timing decorators** are classic real-world examples of reusable cross-cutting functionality.
- **Chaining decorators** applies bottom-up but executes outer-to-inner; without `functools.wraps`, function identity is lost at each layer.
- **Parametrized decorators** (like `retry(n)`) need an extra nesting level to accept configuration arguments.
- **@property / @x.setter / @x.deleter** are built-in decorators that let computed logic hide behind plain attribute syntax.

R. Revision Sheet

Closure

An inner function that remembers variables from its enclosing outer function's scope, even after the outer function has returned.

Decorator

A function that accepts another function, wraps it with additional behavior, and returns a new callable — applied via the `@decorator` syntax.

`*args / **kwargs in wrapper()`

Forwards any positional/keyword arguments to the original function, making a decorator work with any function signature.

Class-based decorator

Uses `__init__` to store the original function and `__call__` to define the wrapper's behavior.

`functools.wraps`

Preserves `__name__` and `__doc__` of the original function on the wrapper, critical for chained decorators and logging.

Parametrized decorator

A decorator factory with three nested levels: outer function for config args, decorator function for the target function, wrapper for the call.

`@property`

Built-in decorator that exposes a method as a read-like attribute; paired with `.setter` and `.deleter` for full attribute control.

I. Interview Questions

- Q1.** What is a closure, and how does it relate to decorators?
- Q2.** Explain what a decorator is and why it does not modify the original function's source code.
- Q3.** Why does a decorator's wrapper function typically use `*args` and `**kwargs`?
- Q4.** What is the difference between a function-based decorator and a class-based decorator?
- Q5.** When you stack multiple decorators on a function, what determines the order in which they execute?
- Q6.** What problem does `functools.wraps` solve, and what breaks if you omit it?
- Q7.** How would you write a decorator that accepts its own configuration argument, such as a retry count?
- Q8.** What is the purpose of the `@property` decorator, and how do `@x.setter` and `@x.deleter` extend it?
- Q9.** Give two real-world use cases where decorators are commonly applied in Python codebases or frameworks.
- Q10.** What would happen if you called a decorated function that takes two arguments, but the decorator's wrapper was defined with no parameters?

F. Further Reading

- Python official documentation — `functools` module (covers `functools.wraps` and related utilities).
- Python official documentation — the `property()` built-in function.
- Python Language Reference — Compound statements section covering function definitions and the decorator (`@`) syntax.
- PEP 318 — Decorators for Functions and Methods (the original proposal that introduced decorator syntax).